

Popular machine learning algorithms in Python

Algorithm	Type	Use case	Advantages	Disadvantages
Linear Regression	Regression	Predicting continuous values (e.g., house prices, sales)	Simple, interpretable, fast, and effective for linear relationships.	Assumes a linear relationship, prone to underfitting for non-linear data.
Logistic Regression	Classification	Binary classification (e.g., spam detection, medical diagnosis)	Easy to implement, interpretable, works well for small datasets.	Limited to binary outcomes, doesn't handle complex relationships well.
Decision trees	Classification/Regression	Predicting outcomes based on decision rules (e.g., credit scoring, disease diagnosis)	Easy to understand, non-linear, no need for feature scaling.	Prone to overfitting, sensitive to noisy data.
Random Forests	Classification/Regression	Ensemble method for more accurate predictions (e.g., fraud detection, customer churn prediction)	Reduces overfitting, handles non-linear data well, robust to noise.	Can be computationally expensive, harder to interpret.
K-Nearest Neighbors (KNN)	Classification/Regression	Instance-based learning (e.g., recommendation systems, handwriting recognition)	Simple to understand, no training phase, effective for smaller datasets.	Computationally expensive for large datasets, sensitive to irrelevant features.
Support Vector Machines (SVM)	Classification/Regression	Classifying complex data (e.g., text categorization, image recognition)	High accuracy, works well in high-dimensional spaces, effective with small datasets.	Computationally expensive, requires proper parameter tuning.
K-Means	Clustering	Clustering data into groups (e.g., customer segmentation, image compression)	Simple, fast, works well for large datasets, easy to implement.	Assumes spherical clusters, sensitive to initial centroids, requires specifying k.
DBSCAN	Clustering	Density-based clustering (e.g., anomaly detection, spatial data analysis)	Can identify clusters of arbitrary shape, robust to noise, no need to specify the number of clusters.	Struggles with clusters of varying densities, sensitive to distance metric.

Fitting the model

```
model.fit(X_train_scaled, y_train)
```

Model training

Once the model has been selected, it must be trained. During this phase, the model learns from the training data by looking for relationships or patterns between the features and the target variable. For example, in a classification problem, the model attempts to predict the class label from the features available for the input.

Training uses the training dataset, where the parameters of the model are adjusted to reduce the value of the error or loss function. The model 'learns' by walking through the data and adjusts its parameters (weights) by using some familiar techniques like gradient descent or other optimisation techniques.

```
#Training the model on the training data
```

```
model.fit(X_train_scaled, y_train)
```

Evaluation of the model

Post the training, it becomes imperative to evaluate the model on data never seen before. This task is undertaken by the test set, which has not been part of the training process. Evaluation helps determine how well the model is generalised for the unseen data representing the real world.

```
from sklearn.metrics import accuracy_score, confusion_matrix
Predicting on the test set
y_pred = model.predict(X_test_scaled)
Evaluating model performance
accuracy = accuracy_score(y_test, y_pred) print(f'Accuracy:
{accuracy * 100:.2f}%')
Confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred) print(conf_
matrix)
```